

What are Bounds in Unity?

In Unity, **Bounds** is an axis-aligned bounding box (AABB) — a box always aligned to world axes (X, Y, Z) that fully contains an object.

Core properties

- **center** — geometric center of the box in world or local space
- **extents** — half the size along each axis (distance from center to edge)
- **size** — full size of the box (equals extents * 2)
- **min** / **max** — corner points with minimum and maximum coordinates

Where Bounds appear

- **Renderer.bounds** — visual bounds in world space; useful for camera visibility checks
- **Mesh.bounds** — mesh bounds in local space; do not change when the object moves or rotates
- **Collider.bounds** — collider bounds in world space; used for fast collision checks

Useful methods

```
Contains(Vector3 point)           // checks if a point is inside the bounds
Intersects(Bounds other)         // checks if two bounding boxes overlap
Encapsulate(Vector3 / Bounds)    // expands bounds to include a point or another
Bounds
Expand(float amount)             // symmetrically expands bounds along all axes
```

Key limitation

Because Bounds is always axis-aligned, rotating a long narrow object by 45 degrees will significantly increase its **Renderer.bounds** to keep containing the object while staying parallel to world axes.

For 2D tasks, use **Rect** (x, y, width, height) instead — **Bounds** is always three-dimensional.

What is Instantiate in Unity and how does it work?

Instantiate clones an existing object at runtime. It is most commonly used to dynamically spawn prefabs: bullets, enemies, effects, or level elements.

Syntax

```
// Full form: object, position, rotation, parent
Instantiate(prefab, position, rotation, parent);
```

- **prefab** — reference to the object to clone
- **position** — Vector3 in world space
- **rotation** — Quaternion; use **Quaternion.identity** to keep the original rotation
- **parent** — (optional) Transform that the new instance becomes a child of

Basic usage example

```
using UnityEngine;
public class Spawner : MonoBehaviour
{
    [SerializeField] private GameObject bulletPrefab;

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Space))
            Instantiate(bulletPrefab, transform.position, transform.rotation);
    }
}
```

Important notes

- **Return value** — the method returns a reference to the created object, allowing immediate property changes or **GetComponent** calls
- **Typed instantiation** — if you pass a component reference, **Instantiate** returns a reference to that same component on the new object, avoiding a **GetComponent** call
- **Performance** — frequent **Instantiate** / **Destroy** calls add CPU overhead and trigger garbage collection pauses. For frequently spawned objects (bullets, particles), use an Object Pool instead
- **Async** — since Unity 2023.3, **InstantiateAsync** is available to create objects in a background thread, avoiding FPS drops when loading complex prefabs

What is Vector3.Lerp() in Unity?

Vector3.Lerp (Linear Interpolation) smoothly finds a point between two vectors A and B based on a coefficient t.

Syntax

```
Vector3.Lerp(Vector3 a, Vector3 b, float t);
```

- **a** — start point (when t = 0)
- **b** — end point (when t = 1)
- **t** — value from 0.0 to 1.0; at t = 0.5 returns the exact midpoint

Rules

- **Clamping** — standard `Lerp` automatically clamps `t` to `[0, 1]`; passing `t = 2.0` still returns point `b`. Use `Vector3.LerpUnclamped` to go beyond
- **Typical uses** — smooth object movement, camera follow, color or size transitions

Code examples

Move from A to B in exactly 2 seconds:

```
float elapsedTime = 0f;
float duration = 2f;

void Update()
{
    if (elapsedTime < duration)
    {
        elapsedTime += Time.deltaTime;
        transform.position = Vector3.Lerp(startPoint.position, endPoint.position,
elapsedTime / duration);
    }
}
```

Smooth follow (camera-style):

```
public Transform target;
public float lerpSpeed = 5f;

void Update()
{
    transform.position = Vector3.Lerp(transform.position, target.position,
Time.deltaTime * lerpSpeed);
}
```

Lerp vs MoveTowards

- `Vector3.Lerp` — speed depends on distance; slows down near the target
- `Vector3.MoveTowards` — moves at a constant speed in units per second; better for physics-driven objects

How does lerpSpeed work when passed as Time.deltaTime * lerpSpeed to Lerp?

The third argument `t` in `Lerp` is a percentage (0 to 1), not a time value. Using `Time.deltaTime * lerpSpeed` is a deliberate trick.

How classic Lerp works

If you increment t from 0 to 1 manually each frame at a fixed rate, the object moves at constant speed and arrives in a predictable time.

The `Time.deltaTime * lerpSpeed` trick

Instead of tracking t , each frame you ask: "where am I now?" and move a percentage of the remaining distance toward the target.

- `Time.deltaTime` — a very small number (e.g. 0.016 at 60 FPS)
- `lerpSpeed` — the "attraction coefficient"

`Lerp(currentPosition, target, 0.1f)` means: "this frame, move 10% of the remaining distance."

Why this creates smooth deceleration

```
Frame 1: distance = 100. 10% of 100 = 10 units moved.  
Frame 2: distance = 90.  10% of 90  = 9 units moved.  
Frame 10: distance = 10. 10% of 10  = 1 unit moved.
```

The object accelerates toward the target quickly and then gently sticks to it — a spring-like or "buttery" effect.

Summary

- Want to arrive in exactly 2 seconds? — manually increment t from 0 to 1
- Want smooth follow (e.g. camera)? — use `Time.deltaTime * lerpSpeed`; here `lerpSpeed` is not time, it is a catch-up coefficient

Note: mathematically, Lerp with this approach never fully reaches 100% of the target (always 10% of the remainder), but Unity rounds the values at some point.

Why do some edges visually intersect after Delaunay triangulation even though the graph should be planar?

The problem was not in the Delaunay algorithm itself, but in how neighbors were interpreted.

In `Delaunator`, the `Triangles` array stores triangle vertices — not a global list of valid edges.

The incorrect assumption was:

```
"two vertices were in the same triangle"  
=  
"there is definitely a valid edge between them"
```

In reality, triangles are defined locally, and some neighbor combinations can be misinterpreted. To get correct edges, use `halfedges`.

How do I use halfedges in Delaunator to get unique edges?

`halfedges` is an array where each oriented edge maps to the index of its opposite oriented edge.

For a triangle with vertices (a, b, c), three half-edges are created: (a,b), (b,c), (c,a).

```
halfedges[e] = h // e and h are opposite sides of the same edge
halfedges[e] == -1 // edge is on the boundary
```

To iterate over unique edges:

```
for (int e = 0; e < delaunator.Triangles.Length; e++)
{
    int opposite = delaunator.Halfedges[e];

    if (opposite == -1 || e < opposite)
    {
        int p = delaunator.Triangles[e];
        int q = delaunator.Triangles[NextHalfedge(e)];
        // unique edge p-q
    }
}
```

How do I convert a List into a vertex adjacency graph?

Use a dictionary keyed by vertex coordinates:

```
Dictionary<(float x, float y), Vertex>
```

Example:

```
var map = new Dictionary<(float, float), Vertex>();

foreach (IEdge e in edges)
{
    var pKey = ((float)e.P.X, (float)e.P.Y);
    var qKey = ((float)e.Q.X, (float)e.Q.Y);

    if (!map.ContainsKey(pKey))
        map[pKey] = new Vertex(...);

    if (!map.ContainsKey(qKey))
```

```
map[qKey] = new Vertex(...);

map[pKey].AddToAllNeighbours(map[qKey]);
map[qKey].AddToAllNeighbours(map[pKey]);
}
```

What algorithms exist for finding maximum flow in a graph?

Algorithm	Idea
Ford-Fulkerson	Find augmenting paths (DFS)
Edmonds-Karp	BFS version of Ford-Fulkerson
Dinic	BFS layered graph + DFS blocking flow
Push-Relabel	Local flow pushing
Min-Cost Max-Flow	Max flow with minimum cost

For games and interactive tasks, Dinic is usually the best choice.

How does Dinic's algorithm work?

Dinic builds a layered graph with BFS, then finds a blocking flow with DFS. These two steps repeat until no more augmenting paths exist.

Core data structure

```
class Edge
{
    int To;
    int Capacity;
    int Flow;
    Edge ReverseEdge;
}
```

Main idea

```
BFS → build layered graph
DFS → push blocking flow through it
repeat until no path from source to sink exists
```

The layered graph ensures each DFS only explores edges that bring flow closer to the sink, giving an overall complexity of $O(V^2 \times E)$.

How should I store the graph after triangulation to track flow per edge and vertex?

Store the graph as:

```
class Vertex
{
    List<EdgeWithFlow> Edges;
    int IncomingFlow;
}

class EdgeWithFlow
{
    Vertex From;
    Vertex To;
    int Capacity;
    int CurrentFlow;
}
```

A vertex's flow value is computed as the sum of its incoming edges' current flow.

How can I make a flow-based puzzle game more interesting?

Mechanics ideas

Edge overheating — edges break when overloaded, forcing the player to redistribute flow.

Time waves — flow arrives in pulses; the player must handle surges.

Edge cost — the player minimizes **max flow + total cost**, turning the puzzle into a simplified min-cost max-flow problem.

Colored flows — different resource types conflict on shared edges.

Gameplay loop

1. Player receives a graph with sources and sinks.
2. Sources generate flow each turn.
3. Player routes flow through edges.
4. Bottlenecks cause overloads.
5. Player reroutes to fix them.
6. Each level adds new constraints: cost, heat, timers, dynamic network changes.

Why does FlowPrefab extend beyond edgePrefab bounds in Unity?

The issue was caused by nested scale values interacting unexpectedly. Key insight:

```
localScale is a multiplier, not a world-space length
```

Specifically: cylinder axis orientation, parent rotation, and child localScale all compound. A child with `localScale.y = 1` does not mean "1 unit long in world space" — it means "same scale as parent along that axis."

Correct prefab hierarchy

```
Edge
├── TubeVisual
├── FlowRoot
│   └── FlowMesh
```

Rules:

```
mesh scale    = 1          (base geometry, unchanged)
parent scale  = edge length (set at runtime)
child scale   = animation  (driven by animation or script)
```

This separates concerns: geometry stays neutral, the parent controls world size, the child handles visual effects.

Why do objects turn pink/purple after switching to URP?

Pink/purple means Unity cannot compile the shader. Common causes:

- Shader does not support URP (was written for Built-in pipeline)
- Old Built-in materials not converted
- Shader Graph node missing the Universal target
- Shader compile errors

Fix

Edit → Rendering → Materials → Convert Built-in Materials to URP

For Shader Graph shaders: open the graph, click the Master Stack, and make sure the target is set to **Universal** (not Built-in).

Why does Alpha Clip Threshold not work correctly in Shader Graph?

In Shader Graph:

- **Alpha** must receive the correct transparency value from a texture or calculation
- **AlphaClipThreshold** compares the **Alpha** value against this threshold — pixels below it are discarded
- If **Alpha** is always close to zero, everything gets clipped regardless of threshold

Check that the Alpha input is connected to the actual alpha channel of your texture, not left at the default 1 or 0.

What is the difference between Lit and Unlit Shader Graph?

Type	Behaviour
Unlit	Ignores scene lighting entirely
Lit	Responds to URP/HDRP lighting

For stylized or cartoon visuals (water, glow effects), Unlit + manually authored highlights is usually preferred — it gives full control over the look without lighting interference.

Why does VS Code show an SDK-style project error with Unity?

Unity uses the old **.csproj** format, which is not SDK-style. The **C# Dev Kit** extension is designed for:

- ASP.NET
- .NET Core
- Modern SDK-style projects

It is not compatible with Unity's project format. For Unity development, use the **C# (Microsoft)** extension alone, without C# Dev Kit installed.

VS Code log interpretation

A healthy Unity + VS Code setup shows:

```
✓ project loaded
✓ language server running
⚠ Firebase x86 warnings ← non-critical, safe to ignore
```

The x86 Firebase warnings appear because the Firebase SDK ships native libraries for multiple architectures. They do not affect editor or desktop play mode.

How do I make UI elements resize with their parent using Vertical Layout Group?

Enable these two options on the **Vertical Layout Group** component:

```
Control Child Size      ✓
Child Force Expand      ✓
```

Also:

- Remove any fixed **Width** / **Height** values on child RectTransforms
- Set anchors to stretch (not corner-anchored) if the child should fill the available space

How do I round the corners of a Unity UI Image?

Best approach: **Sliced Sprite + 9-slicing**

1. Create a sprite with rounded corners in an image editor
2. Set the sprite's **Border** values in the Unity Sprite Editor to define the 9-slice regions
3. Set the Image component's **Image Type** to **Sliced**

Alternatives:

- **Mask** component with a rounded mask sprite
- **Shader Graph** with a custom SDF rounding node
- **UI Toolkit** (has built-in border-radius support)

How do I export a frame from Figma for use in Unity?

1. Select the frame in Figma
2. In the right panel, open the **Export** section
3. Choose format:
 - **PNG** — for UI elements and sprites
 - **JPG** — for backgrounds (lossy, smaller file)
 - **SVG** — for vector elements
4. Click **Export**

For Unity: use **PNG at 2x** to keep sharp edges on high-DPI screens. Import the file, then set **Texture Type** to **Sprite (2D and UI)** in the Inspector.

Why does my sound not play in game?

The most common causes with AudioSource:

1. No audio clip assigned

The **AudioClip** field in the Inspector is empty (in the scene file this appears as `m_audioClip: {fileID: 0}`). Without a clip there is nothing to play. Fix: drag the audio file into the **AudioClip** field.

2. Play On Awake is disabled

If **Play On Awake** is off, the sound will not start automatically when the scene loads. Either enable the flag, or call `audioSource.Play()` manually from a script.

3. Volume is zero

Check that the **Volume** field on the AudioSource is not zero, and that any connected AudioManager group is not muted.

In this project both the first and second issues were present simultaneously: the clip was not assigned and **Play On Awake** was disabled on one of the sources.

Does setting the volume slider to minimum actually mute the audio? If not, how can I fix it?

No. Mapping the slider value (0.0–1.0) directly to `AudioMixer.SetFloat` does not mute at zero. The AudioManager works in decibels (dB), which is a logarithmic scale. A linear value of 0.0 corresponds to $-\infty$ dB, but when passed directly it becomes 0 dB — full volume.

Correct approach

Convert the linear slider value to decibels manually:

```
float LinearToDb(float value)
{
    return value > 0f ? Mathf.Log10(value) * 20f : -80f;
}
```

`-80f` is used as practical silence since AudioManager does not accept $-\infty$.

Usage

```
audioMixer.SetFloat("MasterVolume", LinearToDb/slider.value));
```

At `slider.value == 0` the mixer receives -80 dB (silence). At `slider.value == 1` it receives 0 dB (unchanged volume).

What is Poisson Disk Sampling and why is it used as a comparison for Alg1?

Poisson Disk Sampling is a point placement algorithm that guarantees no two points are closer than a specified minimum distance. The result looks uniformly but randomly distributed — no clusters and no regular grid.

Why it is a relevant comparison

In level generation, Poisson Disk Sampling solves the same problem as Alg1: place vertices on the field so they do not overlap and cover the space evenly. That makes it a direct competitor for the comparative analysis.

Why a regular grid is preferable here

Alg1 uses a regular grid with a fixed step, giving guaranteed $O(n)$ complexity and a predictable field structure. Poisson Disk Sampling is more complex to implement, requires tuning the minimum distance parameter, and does not guarantee full coverage with a small number of points. For a mobile game with deterministic generation, a regular grid is the more reliable and controllable solution.

Can the Alg1 complexity be improved? Are there any bugs in the DFS implementation?

Complexity

The overall complexity of Alg1 is $O(n)$, where n is the number of vertices. This is optimal: every vertex must be visited at least once, so the asymptotic complexity cannot be improved without changing the problem itself.

Bugs found in DFS (LevelCreator.cs)

1. Parent dictionary overwrite

The line `parent[neighbor] = current` executes for every neighbor regardless of whether that neighbor was already visited. If the neighbor already exists in `parent`, its parent gets overwritten, breaking the spanning tree structure.

Fix:

```
if (!parent.ContainsKey(neighbor))
```

```
parent[neighbor] = current;
```

2. Redundant stack.Push

`stack.Push(neighbor)` is called even when the neighbor is already in `parent`, causing vertices to be processed multiple times and potentially duplicating edges in the tree.

Fix: wrap both the `parent` assignment and the `Push` inside the same `ContainsKey` check.